

TrustVerify

A CLI Tool for File Integrity & Digital Signatures

Mini Project I | OSTIM Technical University | 2025–2026

Team Argus | Demir Ege Ortaç 230208045 | Yunus Emre Varol 230208028

1. Why Hashing Alone Is Not Enough to Prove Identity

Hashing is a fundamental tool in cryptography that maps any input file to a fixed-length output called a digest. The SHA-256 algorithm, used in this project, produces a 256-bit hash that is deterministic, collision-resistant, and one-way. If even a single byte of the file changes, the resulting hash will be completely different — this property is called the *avalanche effect*.

The limitation: integrity without authenticity

While hashing proves that a file has not been *accidentally corrupted*, it cannot prove *who created or sent it*. Consider the following attack scenario:

An attacker intercepts the **metadata.json** file. They modify its contents to point to tampered files, then recompute the SHA-256 hashes of those tampered files and replace them in the manifest. The recipient performs a hash check — everything appears valid. But the files have been substituted entirely.

In this scenario, the receiver has no way to know whether the manifest came from the original sender or from an adversary. Hashing provides **data integrity** (was this file altered?), but not **authenticity** (did this file come from who I think it did?).

Summary

Hash alone answers: "*Has this file changed?*"

Hash does NOT answer: "*Who created or sent this file?*"

To answer the second question, we need asymmetric cryptography — specifically, digital signatures.

2. How the Private/Public Key Relationship Ensures Non-Repudiation

RSA (Rivest–Shamir–Adleman) is an asymmetric encryption algorithm that uses a mathematically linked key pair: a **private key** and a **public key**. These keys are generated together, but knowing the public key does not allow anyone to derive the private key.

The signing mechanism

In TrustVerify, the sender performs the following steps:

1. Generate a SHA-256 hash of **metadata.json**
2. Encrypt that hash using the **private key** — this produces the digital signature
3. Distribute: the files, manifest, signature, and **public key** to the receiver

The verification mechanism

The receiver, using only the public key and the received files, performs:

1. Recompute the SHA-256 hash of the received **metadata.json**
2. Use the **public key** to verify that the signature matches that hash
3. If verification passes: the manifest is authentic and unaltered

Non-repudiation explained

Non-repudiation means the sender *cannot later deny* having signed the manifest. Since the private key is kept secret by the sender alone, only they could have produced a signature that is verifiable with their public key. If verification succeeds, it is cryptographically proven that the sender signed that exact manifest — they cannot claim otherwise.

If an attacker modifies the manifest after it has been signed, the hash changes. The old signature will no longer match, and verification will fail — revealing the tampering. Unless the attacker has the sender's private key (which is never shared), they cannot forge a valid signature.

*This project was implemented using Python's **hashlib** library for SHA-256 hashing and the **cryptography** library for RSA key generation, signing (PSS padding), and verification.*